NATO DUE VOLTE

73ª edizione del Carnevale di Massafra

Manuale di Installazione

Installation Manual

Sistema di controllo delle mani scenografiche robotizzate

Robotic Hand Control System

Versione 1.0 · 2025

github.com/mdmmt05/Sistema-elettronico-per-mani-scenografiche

1. Introduzione

Il presente documento descrive le istruzioni di installazione e configurazione del sistema di controllo delle mani scenografiche sviluppato per il carro “Nato Due Volte”, presentato alla 73ª edizione del Carnevale di Massafra (Ass. Crazy Events).

Il sistema è composto da due mani scenografiche robotizzate, ciascuna dotata di cinque dita ad attuazione indipendente. Le mani sono gestite da due unità di controllo dedicate e comandate in modalità wireless tramite un telecomando palmare personalizzato.

1.1 Panoramica del sistema

Progetto	Nato Due Volte — 73ª edizione Carnevale di Massafra
Sistema	Controllo wireless di mani robotizzate
Unità di controllo	2 × Arduino Mega 2560 su shield personalizzata
Telecomando	1 × Arduino Nano su shield personalizzata
Modulo RF	nRF24L01PA+LNA (2,4 GHz, 250 kbps, canale 110)
Attuatori	5 servomotori per mano (10 in totale)
Firmware	controller_code.ino / code_telecomando.ino

1.2 Prerequisiti

Software

- Arduino IDE (qualsiasi versione recente)
- Libreria RF24 — installabile tramite il Library Manager dell’Arduino IDE

Hardware

- 2 × unità di controllo (Arduino Mega + shield personalizzata + nRF24L01+)
- 1 × telecomando (Arduino Nano + shield personalizzata + nRF24L01+)
- 10 × servomotori (5 per mano), con connettore a tre fili (DATA / 5 V / GND)
- 1 × alimentatore regolato 5 V, minimo 5 A, per ciascuna unità di controllo
- 1 × alimentatore regolato 5 V (es. powerbank) per il telecomando

2. Telecomando

Il telecomando è il dispositivo diverso dagli altri due nella dotazione. Ospita un Arduino Nano su una shield personalizzata e dispone di sette pulsanti per l'invio dei comandi di movimento alle due unità di controllo.

2.1 Identificazione

Il telecomando si identifica come il dispositivo morfologicamente diverso rispetto alle due unità di controllo. Espone una porta USB-C sul lato, utilizzata per l'alimentazione.

2.2 Alimentazione

Il telecomando deve essere alimentato tramite la porta USB-C con una sorgente regolata a 5 V (es. powerbank).

> **⚠ WARNING: Non utilizzare mai una sorgente di alimentazione al di fuori dell'intervallo 4,8 V – 5,2 V. Tensioni superiori a questo limite possono danneggiare irreversibilmente la scheda.**

L'alimentazione corretta è confermata dall'accensione di un LED verde in prossimità della porta USB-C.

2.3 Layout e indirizzamento dei pulsanti

I sette pulsanti sono indicizzati in ordine di lettura (da sinistra a destra, dall'alto verso il basso), a partire da 0:

```
[ 0 ]  [ 1 ]  [ 2 ]  [ 3 ]
    [ 4 ]  [ 5 ]  [ 6 ]
```

2.4 Protocollo di comunicazione

Un comando viene trasmesso tramite una sequenza di due pressioni:

- La prima pressione seleziona l'indice del comando per il controller #1.
- La seconda pressione seleziona l'indice del comando per il controller #2.
- Le due pressioni devono avvenire entro una finestra di timeout di 2 secondi; se il timeout scade, la sequenza si azzera e la prima pressione deve essere ripetuta.

Il payload trasmesso via radio è una stringa nel formato "primoIndice,secondoIndice" (es. "2,5"). Ciascun controller estrae solo il valore di propria competenza in base al flag USE_FIRST nel proprio firmware (vedere Sezione 4.2).

2.5 Antenna RF

L'antenna del telecomando dovrebbe essere orientata verso le unità di controllo per ottimizzare la qualità del segnale. In pratica, grazie alla configurazione radio resistente ai disturbi, un puntamento preciso non è strettamente necessario. La latenza massima attesa tra l'invio del comando e la sua esecuzione sulle unità di controllo è di 75 ms. Ritardi superiori indicano con alta probabilità una trasmissione fallita; in tal caso è sufficiente reinviare il comando.

2.6 Smontaggio (se necessario)

Lo smontaggio del telecomando è necessario solo qualora la scheda Arduino Nano interna debba essere riprogrammata.

⚠ WARNING: Devono essere svitate esclusivamente le due viti piccole (M2). La vite più grande è una vite di registro che consente alla piastra inferiore di ruotare attorno al proprio asse per accedere al vano interno — non va rimossa.

3. Unità di Controllo (Mani)

Le due unità di controllo sono assiemi identici, ciascuno costituito da un Arduino Mega 2560 montato su una shield personalizzata. Ogni unità aziona una mano robotizzata tramite cinque servomotori e riceve i comandi wireless dal telecomando.

3.1 Alimentazione

Ciascuna unità di controllo è alimentata tramite il connettore contrassegnato J1 sulla shield personalizzata. Tale connettore si trova in prossimità del condensatore elettrolitico (grosso cilindro argentato).

Connettore	J1
Pin con simbolo quadrato	GND (massa)
Altro pin	VCC (alimentazione positiva)
Tensione richiesta	5 V regolati
Corrente minima	5 A (per garantire sufficiente corrente a tutti e cinque i servomotori simultaneamente)

> ⚠ **WARNING: L'utilizzo di un alimentatore con corrente insufficiente può causare comportamenti anomali dei servomotori o reset del sistema.**

3.2 Modulo RF e antenna

Il modulo RF nRF24L01+ è montato sulla shield ed è dotato di antenna esterna. Per una ricezione ottimale, l'antenna dovrebbe essere fatta uscire dall'involucro metallico della mano.

> **Note:** *Qualora fosse necessario riposizionare l'antenna, è consigliabile spostare solo l'elemento antenna, non l'intero modulo RF, per preservare l'integrità del segnale.*

3.3 Collegamento dei servomotori

I servomotori vanno collegati ai connettori da J3 a J7 sulla shield. L'ordine di cablaggio è indicato a lato di ciascun connettore sulla shield stessa:

DATA — 5 V — GND

L'indice del servomotore nel firmware corrisponde direttamente all'etichetta del connettore:

Connector (shield)	Servo index (firmware)	Finger
J3	0	Pollice
J4	1	Indice
J5	2	Medio
J6	3	Anulare
J7	4	Mignolo

> **Note:** *L'indice 0 corrisponde al primo dito attuato (pollice); gli indici aumentano in sequenza attraverso la mano.*

4. Software

4.1 Panoramica dei file

controller_code.ino	Firmware per le due unità di controllo (Arduino Mega).
code_telecomando.ino	Firmware per il telecomando (Arduino Nano).

4.2 Caricamento del firmware sulle unità di controllo

Entrambe le unità di controllo utilizzano lo stesso file sorgente (controller_code.ino). La costante preprocessore USE_FIRST determina quale valore della stringa di comando ricevuta ciascuna unità deve elaborare:

Controller #1	#define USE_FIRST true
Controller #2	#define USE_FIRST false

> **⚠ WARNING: Questo flag deve essere impostato correttamente prima di caricare il firmware su ciascuna scheda. Caricare entrambe le schede con lo stesso valore causerà la risposta di entrambe le mani allo stesso pulsante.**

Procedura di caricamento:

- Aprire controller_code.ino nell'Arduino IDE.
- Impostare USE_FIRST con il valore appropriato per la scheda di destinazione.
- Selezionare "Arduino Mega or Mega 2560" come scheda target.
- Caricare lo sketch.
- Ripetere la procedura per la seconda scheda con il valore opposto di USE_FIRST.

4.3 Caricamento del firmware sul telecomando

Aprire code_telecomando.ino nell'Arduino IDE, selezionare "Arduino Nano" come scheda target e caricare lo sketch. Non è richiesta alcuna configurazione aggiuntiva.

4.4 Struttura della mappa dei comandi

Il firmware memorizza 7 sequenze di movimento predefinite (una per pulsante) nell'array commandMaps. Ogni comando è composto da un array ordinato di Step, un contatore numSteps e un flag repeat.

Ciascuno Step è definito dai seguenti campi:

Field	Type	Description
`servoIndex`	int (0–4)	Index of the servo motor to move.
`targetAngle`	int (°)	Target position in degrees.
`time`	ulong (ms)	Duration of the movement (interpolated) or minimum settling time (natural).

Field	Type	Description
`pause`	ulong (ms)	Delay after this step before the next begins. A non-zero value also terminates the current synchronised group.
`useInterpolation`	bool	true: linear interpolation over time. false: full-speed snap to target.

Gruppi sincronizzati

Gli step vengono raggruppati automaticamente dal firmware. Step consecutivi con pause == 0 vengono eseguiti in parallelo (tutti i movimenti del gruppo partono simultaneamente). Uno step con pause > 0 termina il gruppo corrente: tutti i movimenti del gruppo vengono completati, dopodiché trascorre la pausa specificata prima dell'inizio del gruppo successivo.

Modifica di un comando

Per modificare un comando esistente o aggiungerne uno nuovo, è necessario modificare la voce corrispondente in commandMaps:

- Aggiungere o rimuovere elementi Step all'interno delle parentesi graffe.
- Aggiornare il valore intero numSteps affinché corrisponda al numero effettivo di step nell'array.
- Impostare il booleano repeat a true se il comando deve ripetersi in loop fino all'arrivo di un nuovo comando, o a false per un'esecuzione singola.
- Qualsiasi comando in arrivo interrompe immediatamente quello in esecuzione.

Note: *Angoli di step al di fuori dell'escursione fisica del servomotore (tipicamente 0–270°) verranno limitati dalla libreria servo. Si raccomanda di validare i nuovi movimenti a bassa velocità prima dell'utilizzo in condizioni di esibizione.*

5. Riferimento Rapido

5.1 Lista di verifica pre-installazione

- Alimentatori verificati: 5 V / 5 A per ciascuna unità di controllo, 5 V per il telecomando.
- controller_code.ino caricato con USE_FIRST corretto su ciascun Mega.
- code_telecomando.ino caricato sul Nano.
- Tutti i connettori dei servomotori inseriti negli slot corretti (J3–J7).
- Antenne RF portate all'esterno degli involucri metallici.
- LED verde sul telecomando acceso all'accensione.

5.2 Risoluzione dei problemi

Il telecomando non si accende	Verificare la sorgente di alimentazione USB-C. Controllare che la tensione sia compresa tra 4,8 e 5,2 V.
Comando non ricevuto dall'unità di controllo	Reinviare il comando. Assicurarsi che le due pressioni avvengano entro il timeout di 2 s. Verificare l'orientamento dell'antenna RF.
La mano sbagliata risponde a un pulsante	Verificare che USE_FIRST sia impostato correttamente su ciascuna unità di controllo e ricaricare il firmware se necessario.
Il servo si porta in posizione errata	Controllare l'orientamento del connettore del servo (DATA / 5 V / GND). Verificare l'assegnazione J3–J7.
Movimento anomalo / reset del sistema	Corrente dell'alimentatore insufficiente. Utilizzare un alimentatore con corrente minima di 8 A.
Ritardo tra pressione e movimento della mano	Atteso: fino a 75 ms. Se sistematicamente superato, verificare la presenza di interferenze RF o pressione insufficiente sui pulsanti.

1. Introduction

This document provides the installation and configuration instructions for the robotic hand control system developed for the float “Nato Due Volte,” presented at the 73rd edition of the Carnevale di Massafra.

The system consists of two scenic robotic hands, each equipped with five independently actuated fingers. The hands are driven by two dedicated control units and operated wirelessly via a custom handheld remote controller.

1.1 System overview

Project	Nato Due Volte — 73rd Carnevale di Massafra
System	Wireless robotic hand control
Control units	2 × Arduino Mega 2560 on custom shield
Remote	1 × Arduino Nano on custom shield
RF module	nRF24L01+ (2.4 GHz, 250 kbps, channel 110)
Actuators	5 servo motors per hand (10 total)
Firmware	controller_code.ino / code_telecomando.ino

1.2 Prerequisites

Software

- Arduino IDE (any recent version)
- RF24 library — installable via the Arduino IDE Library Manager

Hardware

- 2 × control unit assembly (Arduino Mega + custom shield + nRF24L01+)
- 1 × remote controller assembly (Arduino Nano + custom shield + nRF24L01+)
- 10 × servo motors (5 per hand), with three-wire connectors (DATA / 5 V / GND)
- 1 × regulated 5 V power supply, minimum 5 A, for each control unit
- 1 × regulated 5 V power supply (e.g. powerbank) for the remote controller

2. Remote Controller

The remote controller is the unique device in the kit. It hosts an Arduino Nano on a custom shield and provides seven push-buttons for sending movement commands to the two control units.

2.1 Identification

The remote controller can be identified as the device that differs from the other two. It exposes a USB-C port on its side, used for power supply.

2.2 Power supply

The remote controller must be powered via its USB-C port with a regulated 5 V source (e.g. a powerbank).

> **⚠ WARNING: Never use a power source outside the 4.8 V – 5.2 V range. Voltages above this threshold may permanently damage the board.**

Correct power delivery is confirmed by a green LED illuminating near the USB-C port.

2.3 Button layout and addressing

The seven buttons are indexed in reading order (left to right, top to bottom), starting from 0:

```
[ 0 ]  [ 1 ]  [ 2 ]  [ 3 ]
    [ 4 ]  [ 5 ]  [ 6 ]
```

2.4 Communication protocol

A command is transmitted using a two-press sequence:

- The first button press selects the command index for controller #1.
- The second button press selects the command index for controller #2.
- The two presses must occur within a 2-second timeout window; if the window expires, the sequence resets and the first press must be repeated.

The payload sent over the air is a comma-separated string of the form “firstIndex,secondIndex” (e.g. “2,5”). Each controller extracts only the value relevant to it based on the USE_FIRST flag in its firmware (see Section 4.2).

2.5 RF antenna

The antenna protruding from the remote controller should be oriented towards the control units for optimal signal strength. In practice, due to the noise-resistant radio configuration, precise pointing is not strictly required. A maximum latency of 75 ms is expected between command transmission and execution on the control units. Delays exceeding this threshold most likely indicate a failed transmission; the command should be resent in that case.

2.6 Disassembly (if required)

Disassembly of the remote controller is required only if the internal Arduino Nano board needs to be reprogrammed.

> **⚠ WARNING: Only the two small M2 screws must be removed. The larger screw is a pivot that allows the bottom plate to rotate and expose the internal compartment — do not unscrew it.**

3. Control Units (Hands)

The two control units are identical assemblies, each consisting of an Arduino Mega 2560 mounted on a custom shield. Each unit drives one robotic hand via five servo motors and receives wireless commands from the remote controller.

3.1 Power supply

Each control unit is powered through the connector labelled J1 on the custom shield. This connector is located near the electrolytic capacitor (large silver cylindrical component).

Connector	J1
Square-marked pin	GND (ground)
Other pin	VCC (positive supply)
Required voltage	5 V regulated
Minimum current	5 A (to ensure adequate power to all five servo motors simultaneously)

> ⚠ **WARNING: Using a power supply with insufficient current rating may cause erratic servo behaviour or system resets.**

3.2 RF module and antenna

The nRF24L01+ RF module is mounted on the shield and is fitted with an external antenna. For optimal reception, the antenna should be routed outside the metallic enclosure of the hand.

> **Note:** *If repositioning of the antenna is required, only the antenna element itself should be relocated. Moving the entire RF module may compromise signal integrity.*

3.3 Servo motor connections

Servo motors are connected to the connectors labelled J3 through J7 on the shield. The wiring order is printed on the shield next to each connector:

```
DATA — 5 V — GND
```

The servo index in the firmware corresponds directly to the connector label:

Connector (shield)	Servo index (firmware)	Finger
J3	0	Thumb
J4	1	Index
J5	2	Middle
J6	3	Ring
J7	4	Little

> **Note:** *Servo index 0 corresponds to the first activated finger (thumb); indices increase sequentially across the hand.*

4. Software

4.1 File overview

controller_code.ino	Firmware for the two control units (Arduino Mega).
code_telecomando.ino	Firmware for the remote controller (Arduino Nano).

4.2 Flashing the control units

Both control units use the same source file (controller_code.ino). The USE_FIRST preprocessor constant determines which value from the received command string each unit responds to:

Controller #1	#define USE_FIRST true
Controller #2	#define USE_FIRST false

⚠ WARNING: This flag must be set correctly before flashing each board. Flashing both boards with the same value will cause both hands to respond to the same button press.

To flash a control unit:

- Open controller_code.ino in the Arduino IDE.
- Set USE_FIRST to the appropriate value for the target board.
- Select “Arduino Mega or Mega 2560” as the board.
- Upload the sketch.
- Repeat for the second board with the opposite USE_FIRST value.

4.3 Flashing the remote controller

Open code_telecomando.ino in the Arduino IDE, select “Arduino Nano” as the board, and upload the sketch. No configuration is required.

4.4 Command map structure

The firmware stores 7 pre-defined movement sequences (one per button) in the commandMaps array. Each command consists of an ordered array of Steps, a step count (numSteps), and a repeat flag.

Each Step is defined by the following fields:

Field	Type	Description
`servoIndex`	int (0–4)	Index of the servo motor to move.
`targetAngle`	int (°)	Target position in degrees.
`time`	ulong (ms)	Duration of the movement (interpolated) or minimum settling time (natural).
`pause`	ulong (ms)	Delay after this step before the next begins. A non-zero value also terminates the current synchronised group.

Field	Type	Description
`useInterpolation`	bool	true: linear interpolation over time. false: full-speed snap to target.

Synchronised groups

Steps are grouped automatically by the firmware. Consecutive steps with pause == 0 are executed in parallel (all servo movements in the group start simultaneously). A step with pause > 0 terminates the current group, all movements in the group complete, and the specified pause elapses before the next group begins.

Modifying a command

To modify an existing command or add a new one, edit the corresponding entry in commandMaps:

- Add or remove Step entries within the braces.
- Update the numSteps integer to match the actual number of steps in the array.
- Set the repeat boolean to true if the command should loop until interrupted, or false for single execution.
- Any incoming command immediately interrupts the currently executing one.

> **Note:** *Step angles outside the physical servo range (typically 0–270°) will be clamped by the servo library. Validate new movements at low speed before use in performance conditions.*

5. Quick Reference

5.1 Pre-installation checklist

- Power supplies verified: 5 V / 5 A for each control unit, 5 V for remote.
- controller_code.ino flashed with correct USE_FIRST on each Mega.
- code_telecomando.ino flashed on the Nano.
- All servo connectors inserted in correct J3–J7 slots.
- RF antennas routed outside metallic enclosures.
- Green LED on remote controller illuminated at power-on.

5.2 Troubleshooting

Remote controller does not power on	Check USB-C power source. Verify voltage is within 4.8–5.2 V.
Command not received by control unit	Re-send the command. Ensure both button presses occur within the 2 s timeout. Check RF antenna orientation.
Wrong hand responds to a button	Verify USE_FIRST is set correctly on each control unit and re-flash if necessary.
Servo moves to incorrect position	Check servo connector orientation (DATA / 5 V / GND). Verify J3–J7 assignment.
Erratic servo movement / system resets	Insufficient power supply current. Use a supply rated at a minimum of 5 A.
Latency between press and hand movement	Expected: up to 75 ms. If consistently exceeded, check for RF interference or insufficient button press duration.